

Finance Radar 组内说明书

我们到底做了个什么东西 —— 给组员看的大白话版

2026-08-16 · 对应代码提交 ceb9f57 · 生产版本 2026.08.15.4 · 本文所有描述均来自实际源码

0. 一句话说清楚

我们做的是个金融事件的"证据雷达"。

它每 5 分钟自动跑一轮，去 13 个官方网站（美联储、SEC、CFTC、FDA、FTC、FDIC、欧洲央行、美国能源署、美国劳工统计局）和 1 个聚合新闻源抓消息；把消息变成"事件"；给每个事件配上能点回原文的精确引文；记录这个事件后来被官方修订、更正、撤回的全过程；并且在证据不够或者互相矛盾的时候，明确地说"我不下结论"。

它不是炒股软件。代码里根本没有下单、持仓、余额这些东西，而且这一点是被数据库约束和 CI 强制保证的，不是靠自觉。

一句话记住产品定位

别人做的是"让 AI 猜涨跌"，我们做的是"把消息怎么变成结论完整保存下来，并且在证据不足时拒绝下结论"。

规模（都能现场核验）

项目	数值	怎么查
生产事件数	13,357 条（其中 7,471 条待核验）	公网总览页
Python 代码	202 个文件，约 66,700 行	<code>app/</code> 13,972 · <code>scripts/</code> 34,153 · <code>tests/</code> 18,575
测试	664 passed / 5 skipped / 20 subtests	<code>python -m pytest -q</code>
API 路由	25 条（1 条根路径 + 24 条 <code>/api/v1/*</code> ）	<code>/docs</code>
Web 页面	1 个总览 + 5 个功能页，4 种身份	<code>app/web/</code>
冻结演示案例	4 个（断网也能跑）	<code>replay/cases/cases.json</code>

1. 一条新闻的一生

这是整个项目最核心的一章。看懂这一章，你就知道我们在做什么了。

谁在跑：`app/workers/continuous.py` 是一个常驻进程，每 `--interval` 秒（默认 300 秒 = 5 分钟）启动一次子进程 `scripts/run_live_cycle.py`。子进程里的 `run_cycle()` 函数就是下面这整套流程，从头到尾顺序执行。

为什么用子进程：一轮采集崩了不会带崩常驻进程。而且父进程在启动子进程之前会先删掉上一轮的报告文件，这样子进程如果启动失败，绝不会读到上一轮的成功报告然后误判成"这轮也成功了"。

1.1 十六个步骤

0. 抢锁。先去 `runtime_leases` 表抢一个叫 `live_cycle` 的租约，TTL 1200 秒，后台线程每 60 秒续一次。抢不到就直接退出，返回码 3，状态记 `SKIPPED`。→ 保证同一时刻只有一个采集周期在跑，两个 worker 不会互相打架写坏数据。
1. 官方源采集。13 个官方 RSS / Atom / API 挨个拉。每条消息存进 `raw_observations` 表（原始观测，不可变）。如果同一条消息内容被改过，就往 `source_revisions` 表追加一条修订记录，类型是 `new / edit / delete` —— 原记录不动。→ 所有 `sec_*` 源需要环境变量 `SEC_USER_AGENT`（SEC 官方要求访问者声明身份），没配就跳过并记一条 `error`。
2. 聚合发现源。OpenNews 三个分类（`macro / ai / web3`）。权威等级只有 `P2_experimental`，也就是"参考线索"级别，单独一条 P2 证据永远不足以让事件成立。
3. 候选抽取。`process_pending(limit=500)`。用 20 条正则规则（破产 / 退市 / 黑客攻击 / 并购 / 财报 / 监管处罚 / 货币政策 / 制裁关税 / 地缘冲突 / 新币上线 / 大规模裁员 / CPI / 非农 / 执法行动 / 产品召回 / 高管变更 / 债务违约 / 增发稀释 / 重组 / 分红回购）把观测变成 `canonical_events` 表里的候选事件（`status = candidate`）。P0/P1 来源优先处理；P2 有积压上限、会过期、会被淘汰。
4. SEC 文件深挖。`enrich_sec_filings(limit=8)`。去拉 8-K、10-K、10-Q、20-F 等表单的正文，抽出精确段落作为证据。
5. 官方原文页面深挖。`enrich_official_primary_pages(limit=20, max_chars=1200)`。
6. 复核排队。算一个 `review_score`，决定人应该先看哪一条。
7. 证据路由。按 `config/live_evidence_routes.json` 给每个事件排"该去哪里找证据"。
8. 人工裁决落库。把 `config/live_primary_adjudications.json` 里人已经做出的判断写回账本。→ 注意这一步的输入是人写的文件，不是模型输出。
9. 原文快照。`archive_evidence_sources(limit=8)`。把官方页面的 HTML/PDF 原始字节按 SHA-256 存成内容寻址对象。规则很严：只走 HTTPS、只允许已注册的官方域名、跳转之后重新校验域名、单个文件不超过 10 MiB、只接受 HTML 和 PDF。越域的连接只记 `policy_skipped`，绕不过去。
10. 影子模型跑分。`run_shadow_batch(scan_limit=200, run_limit=100)`。小模型给事件打标签，只写运维日志，不改任何事件状态。
11. 证据 Agent。`run_pending_evidence_agents(limit=4)`。详见第 3 章。
12. 资产关联。把事件关联到具体资产，并做 SEC 发行人 → 股票代码的映射。
13. 行情观察排期。给 14 天内的事件排 `T+5m / T+30m / T+1d` 三个观察窗口。
14. 行情采集。加密资产走 Binance 公开免认证行情接口，非加密资产走 Twelve Data。宽限期分别是 2 分钟 / 5 分钟 / 30 分钟。错过窗口就写 `MISSED_WINDOW`，绝不拿当前最新价补写历史。
15. 告警出箱。先把超过 24 小时还没发出去的清掉，再挑符合资格的事件入队。

16. 发送。默认 `dry_run`，不加 `--send` 参数一条都不会真的发出去。

1.2 告警资格是硬门槛

不是"模型觉得重要"就能发告警。`qualified_events()` 里的 SQL 要求同时满足五个条件：

```
status = 'verified' AND label_status = 'verified'
AND manual_grade IN ('S','A++','A')      ← 人评的等级，不是模型给的
AND no_trading = 1
AND 事件日期在最近 3 天内
AND 至少有一条带 URL 的证据
```

这条最值得记住

`manual_grade` 是人工评级字段。也就是说：只要没有人手动评过 S / A++ / A，无论模型多确信，这条事件永远发不出告警。模型没有开闸的权力。

1.3 一轮跑完可能是四种结果

状态	什么情况	要不要慌
SUCCESS	子进程返回 0	正常
SKIPPED	返回 3，锁被别人占着	正常，说明上一轮还没跑完
DEGRADED	部分来源失败，但其余来源、证据路由和落库都完成了	不等于挂了。某个官方站点超时或 403 很常见
FAILED	整轮没产出	要查

2. 数据库长什么样

两个 SQLite 文件，故意分开：

- 账本库 `finance_radar.sqlite3` (Schema 12, WAL 模式) —— 研究数据：来源、观测、事件、证据、行情。
- 运维库 `finance_radar_operations.sqlite3` (Schema 6) —— worker 周期、备份记录、模型运行、回放记录、Agent 决策、人工覆盖、盲审样本。

为什么分开：运维噪音（每 5 分钟一条心跳）不应该污染研究账本，而且备份和权限可以分开管。

2.1 账本库的主链条

```
sources          来源登记（含权威等级 P0/P1/P2/P3）
├── raw_observations  原始观测（唯一键 = 来源 + 外部ID，不可变）
├── source_revisions  修订链（new / edit / delete, 只追加）

canonical_events  规范事件（四态：candidate / verified / weak / rejected）
├── event_versions    事件版本，只追加，每次变化留一版 + 变更原因
├── event_observations 事件 ↔ 观测（多对多）
├── event_evidence     证据：URL + 精确段落 + 证据状态
├── event_assessments  人工评级：S/A+/A/B/C + P0~P3 + 六个分项
├── event_chains       事件链：主事件 / 同事件支撑 / 后续版本 /
├── event_chain_members 后果 / 行政控制
├── event_entities     涉及的实体（主体/行为方/目标/地点/监管方）
├── event_asset_impacts 关联资产
├── event_market_metrics 事后行情指标（只作审计）

market_jobs → market_snapshots  行情任务与只读快照
pipeline_jobs / observation_jobs  两级任务队列
alert_outbox → alert_delivery_attempts 告警出箱 + 每次投递记录
runtime_leases 租约锁
```

2.2 六条写死在数据库里的安全约束

这个答辩一定会被问到，一定要能背下来至少三条：

```
canonical_events.no_trading          CHECK (no_trading = 1)
event_evidence.auto_verification_allowed CHECK (= 0)
event_market_metrics.metric_scope     CHECK (= 'post_event_audit_only')
event_market_metrics.allowed_for_discovery_rank CHECK (= 0)
event_market_metrics.allowed_as_model_feature CHECK (= 0)
market_snapshots.read_only           CHECK (read_only = 1)
```

这几行是什么意思

它们是 SQLite 的 `CHECK` 约束，不是注释。意思是：就算有人写错代码，数据库层面也插不进一条"允许交易"或者"行情可以当模型特征"的记录——insert 会直接报错。安全边界是被数据库强制的，不靠代码自觉。

3. AI 到底用在哪

整个系统里只有两个地方是 AI，而且两个都不能决定结论。这一节要能讲得特别清楚。

3.1 风险路由小模型 `app/models/risk_router.py`

- CPU 上跑的 scikit-learn 文本分类器，输出三档：`RISK_REVIEW` / `NON_TARGET` / `ABSTAIN`。
- 模型文件坏了会可见降级成关键词兜底（`runtime = "fallback"`），不会让 API 崩溃。这条有专门的故障注入演练验证过。
- 每次输出都强制带 `shadow: true` 和 `no_trading: true`。
- 每次决策都记一个 `input_sha256`，把文本、证据状态、阈值配置全哈希进去——换了阈值就是另一次决策，不会被静默去重。

它经历过一次彻底的失败和一次彻底的重做

这是本项目最值得讲的一段，三代盲测的真实数字（每一代的盲测集都是先冻结标签、再跑推理，且与训练集零重叠）：

版本	盲测集	准确率	风险召回	普通新闻误报率	门禁
v1	blind-v1 (40 条)	50.0%	100%	95.0%	FAIL
v3	blind-v2 (80 条)	56.2%	40.0%	6.7%	FAIL
v4 (当前生产)	blind-v3 (80 条)	96.3%	96.7%	6.7%	PASS

v1 错在哪：它对 40 条里的 39 条都喊 `RISK_REVIEW`，一次 `NON_TARGET` 都没输出过。误判的全是"官方腔调但零负面内容"的东西——NVIDIA 的游戏上线公告、美联储悼念格林斯潘的讣告、三年期支付研究报告。根因是训练语料做空特化，模型学到的是"官方财经文体 = 风险"，它没有"官方但不是坏事"这个概念。

顺便回答一个很多人会想到的问题：把输出反过来不就对了？

不行，实测过。反过来之后严格准确率从 50.0% 掉到 47.5%，风险召回从 100% 掉到 0%——一个风险雷达漏掉全部真实风险，比误报更糟，只是把"哪道门不过"换了一下。

原因：v1 是退化（对什么都喊 RISK），不是反相关。反转只在 $\text{ROC-AUC} < 0.5$ 时才有意义。v1 的实际 AUC 是 0.857，说明它排序的方向本来就是对的，坏的是决策阈值。

v4 怎么修好的：不是调参，是换架构。v2 失败后模型卡里留了一句教训——`"Evidence state is a structured gate, not a text class."`（证据状态是一道结构化闸门，不是一个文本类别）。于是 v4 拆成三层，代码在 `RiskRouter.predict()` 里能直接看到：

1. 结构化证据门（确定性）—— 证据不是 `PRIMARY_SUPPORTED`，直接 `ABSTAIN`、置信度 1.0，模型根本不被调用
2. 高精度语义策略门（确定性）—— 能定的直接定，只有 `DEFER_TO_MODEL` 才往下走
3. 二分类小模型 —— 只处理前两层都拿不准的那部分

另外还有一道 `risk_scope_gate`（判断这条消息是否落在处理范围内），每次调用都会计算并记入 `call_counts`，在旧架构路径下直接负责拒绝噪音和弃权。

blind-v3 上 `ABSTAIN` 的精度和召回都是 1.000，正是因为 `ABSTAIN` 现在由确定性闸门决定，轮不到模型。把模型的活越切越小，整体准确率反而上来了。

那为什么现在还是 `SHADOW`？——这才是当前真正的缺口

v4 的盲测 11 道门全过，但晋级决策是 `QUALIFIED_SHADOW`（合格但仍在影子态），不是正式判断器。原因写在模型卡里：

```
"label_provenance": "AI_RUBRIC_ADJUDICATOR_NOT_HUMAN"
"human_labels_claimed": false
```

blind-v2 和 blind-v3 的标签是 AI 按评分规则打的，不是人打的。所以我们不承认它有资格出 shadow。

要真正晋级，缺的是人工双盲标注——两人独立标 80 条（`RISK 30 / NON_TARGET 30 / ABSTAIN 20`），互不可见，分歧由第三人仲裁，然后用人工标签冻结全新的 blind-v4。基础设施（Adjudication Studio 页面、v3 标签合同、仲裁流程）已经全部写好，真实人审 0 条。这件事必须两个人做，做完就是最硬的团队协作证据。

有条捷径，千万别走

把 v1 的判定阈值从 0.62 挪到 0.768，就能刚好三道门全部踩线通过（召回 80% / 误报 20% / 准确率 80%）。不许这么干：那是在盲测集上调参，盲测集一旦用来调参就作废了；而且 40 条样本里一条 = 5%，踩线通过意味着换一条样本就掉下来。老师只要问一句“这个阈值你是怎么定的”，整套评测的可信度就没了。

3.2 证据 Agent 的本地小模型 `app/services/evidence_agent.py`

这块设计得比较细，是我们工程上最有说服力的部分。

跑在哪

回环地址（127.0.0.1）上的 llama.cpp，模型是 Qwen2.5-0.5B。代码里 `_is_loopback_url()` 强制检查：不是 loopback 地址直接抛异常。它物理上连不上外网，也没有 API key。

流程

- 1. 从事件摘要里抽出最多 6 条 claim（声明）。
- 2. 从证据里取 exact_excerpt（精确引文）——是原文里抠出来的原话，不是 AI 复述的。
- 3. 用词元重合度把证据连到 claim 上，形成证据图，边有两种：SUPPORTS（支持）和 CONTRADICTS（反驳）。

判定是谁做的：100% 是确定性规则，不是模型

单条 claim 的情况	判定
有 CONTRADICTS 边	HUMAN_REVIEW
有 PO 权威来源的 SUPPORTS 边	PRIMARY_SUPPORTED
只有非 PO 的 SUPPORTS 边	DISCOVERY_SUPPORTED
什么都没有	INSUFFICIENT

整个事件的结论：任何一条冲突 → HUMAN_REVIEW；任何一条不足或只有发现级证据 → INSUFFICIENT；全部被 PO 支撑才 → EVIDENCE_READY。

模型只被允许写一句摘要，而且要过 8 道校验

- 1. 返回的 JSON 必须严格匹配 schema，多一个字段少一个字段都不行
- 2. 摘要不超过 600 字符
- 3. 不能编造 evidence_id，引用必须在已知集合内
- 4. 判定必须跟确定性证据图完全一致，不一致直接拒
- 5. 说 SUPPORTED 就必须引用 SUPPORTS 边，说 CONTRADICTED 就必须引用 CONTRADICTS 边
- 6. 说 INSUFFICIENT 就不许引用任何东西
- 7. 不能出现 EVIDENCE_READY / HUMAN_REVIEW 这类控制词
- 8. 不能出现"买入 / 卖出 / 下单 / 执行交易 / place order / buy now"之类的字样

任何一条不过 → 抛 LocalModelError → 回退到确定性摘要，并把失败码记进审计记录。系统降级是可见的，不是静默的。

收据指纹：防止拿旧结论糊弄

evidence_receipt_fingerprint() 把事件版本号 + 全部证据的关键字段（引文、状态、来源、权威等级、更新时间）哈希成一个指纹。下次要复用旧的 Agent 决策时，先比对指纹：证据变了指纹就变，旧决策自动作废、重跑。

而且更早期没有指纹字段的历史决策，一律默认失效——这就是 fail closed：拿不准的时候往保守走，不往通过走。

3.3 这些地方不是 AI（容易被误会）

候选抽取的 20 条正则规则、证据门、light verification、告警资格判定、行情窗口调度、S/A++/A/B/C 评级——全是确定性代码或人工判断，一行 AI 都没有。

4. 页面和身份

Web 是 Streamlit 写的。同一份代码，靠环境变量 `FINANCE_RADAR_UI_ROLE` 切成四种身份（public / reviewer / operator / admin）。`require_ui_role()` 在页面渲染之前就 `st.stop()` 拦掉越权访问。

页面	文件	谁能看	内容
态势总览	Home.py	所有人	最新事件流、来源健康度、待核验数量
事件工作台	pages/1_Event_Intelligence.py	所有人	单个事件的证据矩阵（权威等级 / 状态 / 精确引文）、修订时间线、只读行情上下文
证据演示	pages/2_Replay_Lab.py	所有人	4 个冻结案例逐步回放
运行与模型	pages/3_Operations_and_Model.py	operator / admin	worker 周期、备份状态、模型卡、盲测结果
双人盲审	pages/4_Adjudication_Studio.py	reviewer / admin	两人独立打标，互相看不到对方答案，也看不到模型结果
方法与边界	pages/5_Method_and_Boundaries.py	所有人	来源、时间、置信度的口径，以及“我们不做什么”

公网只暴露 public。reviewer / operator / admin 三种内部界面都是回环地址、按需启动、各自独立令牌，通过 SSH 隧道访问，公网 Nginx 不暴露 API 和管理路径。

API 权限分三层

依赖	能干什么
require_reviewer	看事件 trace、写人工覆盖记录、双人盲审队列与提交
require_operator	跑证据 Agent、跑回放、切演示模式、看模型状态、看证据存档
require_admin	上面两个的超集，外加创建盲审样本

每个响应都带 `schema_version`（当前 1.1）、`trace_id` 和 `generated_at`，API 层还有 180 次/分钟的限流。

5. 四个冻结回放案例

`replay/cases/cases.json`，由 `ReplayService` 按“事件时钟”逐秒回放。完全不联网，所以答辩当天断网、当天没有真实重大事件，都不影响演示。

#	案例	过程	结果
1	sec_bankruptcy_verified	0 秒：P2 传闻说公司要重组 → 18 秒：P0 官方 8-K Item 1.03 落地	RISK_REVIEW 有告警资格

2	positive_earnings_non_target	P1 发行人宣布业绩创纪录、上调指引	NON_TARGET 证明系统不是"只会喊风险"
3	rumor_correction_abstain	0 秒 : P3 传闻说要破产 → 11 秒 : P1 发行人否认 (contradicts)	ABSTAIN 无告警
4	sec_filing_corrected_abstain	0 秒 : P0 官方 8-K 披露破产 → 42 秒 : P0 更正件撤回该披露	从"有告警资格"退回 ABSTAIN 告警资格被收回

回放里的门禁逻辑（答辩可以直接念）

```
if 有冲突证据:
    最终 = ABSTAIN, 理由 = evidence_conflict_cap
elif 模型说 RISK_REVIEW 但没有 P0 证据:
    最终 = ABSTAIN, 理由 = evidence_gate_hold_pending_primary
else:
    最终 = 模型标签, 理由 = shadow_router_after_evidence_gate

告警资格 = (最终 == RISK_REVIEW) 且 有P0证据 且 无冲突
```

第 4 个案例是我们最强的演示点

它证明了一件别的项目做不到的事：证据可以被官方撤回，我们的结论也就跟着退回去。系统不是只会往前加结论，它也会往回收。

6. 三条红线和它们的保证方式

红线	代码层面怎么保证
不碰交易 NO TRADING	① 数据库 CHECK 约束 <code>no_trading = 1</code> ; ② CI 自动扫描交易写路由; ③ 故障注入演练枚举全部生成的 API 路由, 断言 <code>order / position / balance / brokerage / trade-execution</code> 路径不存在
模型不能自己把事件 升级成 VERIFIED NO AUTO VERIFY	① <code>auto_verification_allowed CHECK (= 0)</code> ; ② Agent 的结果只写运维库, 碰不到账本库的事件状态; ③ 正式状态只能由人工裁决, 或由受限额授权的 <code>light verification</code> 写入
事后行情不能进模型 特征 NO LEAKAGE	三个 CHECK 约束一起把: <code>metric_scope = 'post_event_audit_only'</code> 、 <code>allowed_as_model_feature = 0</code> 、 <code>allowed_for_discovery_rank = 0</code>

另外还有两条操作层面的: Telegram 默认 `dry-run`, 要真发必须显式加 `--send` ; Evidence Agent 的本地模型是可选的、回环的、只做建议的, 部署和恢复流程默认不启用它。

light verification 为什么不算"模型自动核验"

这个模块可以把事件从 `candidate` 提升到 `verified`, 所以它的限制特别死:

- 只对 4 种离散的、能从原始记录直接读出来的事件类型生效: `delisted`、`voluntarydelisting`、`reverse_split`、`bankruptcy_liquidation`。涉及阈值、价格、比率、会计口径的一律不行——那些需要定量解析, 词面匹配不够。
- 必须有一条可寻址的 P0/P1 主来源段落, 同时通过身份、事实、日期、语气 (modality) 四道门。任何一道没过, 不是给出否定结论, 而是保持事件原状 + 建一条"缺哪些证据维度"的后续任务。
- 要写入正式状态, 必须加载一份限额授权合同: 绑定批次 ID、过期时间、最大应用条数, 并且逐个事件绑定当前版本号和证据指纹。指纹对不上就拒绝执行。
- 常驻 worker 里只跑只读报告模式, 代码注释写得很明白: 这里绝不允许放长期授权或 `--apply`。

7. 三个禁飞区要写什么

现状: 一行都还没写

`app/core/` 这个目录目前不存在。三个内核文件全部为空白。这是我们离封版最近的一道坎, 也是 AI 帮不上忙的部分。

共同约束 (三个都一样): 纯函数, 不连网、不连数据库、不调 LLM, 输入输出固定, 每个至少 20 个聚焦测试, 负责人要能脱离 IDE 逐行讲。预计每个 80–150 有效行。

FZ1 · `app/core/event_fingerprint.py` 负责人: _____

干什么: 判断"这条新消息是重复、是修订、还是全新事件"。

- 输入：规范化后的事件字段（来源、标题、摘要、日期、主体、URL）
- 输出：一个稳定指纹 + 关系判定（DUPLICATE / REVISION / NEW）
- 要能讲清的难点：标题被改了一个词，算修订还是新事件？同一件事被两个来源分别报道，怎么合并？指纹碰撞了怎么办？
- 可以参考但不能抄：`scripts/event_ledger.py` 的 `stable_id()`、`source_revisions` 表的 `revision_kind` 字段设计

FZ2 · `app/core/evidence_gate.py` 负责人：_____

干什么：给定一组证据，输出证据门结果。

- 输入：证据列表（权威等级、证据状态、有没有精确引文、彼此是否矛盾）
- 输出：`PRIMARY_SUPPORTED` / `DISCOVERY_SUPPORTED` / `INSUFFICIENT` / `HUMAN_REVIEW` + 理由码
- 要能讲清的核心：为什么必须 fail-closed —— 任何拿不准的情况都往更保守走，而不是往通过走
- 可以参考但不能抄：`EvidenceAgent._status()`、`derive_evidence_context()`

FZ3 · `app/core/finality_gate.py` 负责人：_____

干什么：给定法律阶段和身份冲突，输出最终状态和等级上限。

- 输入：法律阶段（申请 / 受理 / 裁定 / 更正 / 撤回）、身份是否冲突、是否已具最终性
- 输出：状态 + 最高等级上限（S/A++/A/B/C）+ no-trading 不变量
- 要能讲清的核心：哪些状态转换是非法的、官方更正和撤回怎么处理
- 可以参考：`canonical_events.status` 的四态、`provisional_grade_cap` 字段

红线

`risk_router.py`、`replay.py` 这些现有代码是 AI 辅助写的产品脚手架，绝对不能倒签成禁飞区成果。课程规则写明：走查时被发现禁飞区代码是 AI 生成的，必须删掉手写重做，该模块分数清零。这一点 `.agent/forbidden_zones.md` 里已经白纸黑字声明过了。

8. 怎么把它跑起来

```
# 环境 (要求 Python 3.12)
python -m venv .venv
.venv/bin/pip install --require-hashes -r requirements-dev.lock

# 三个进程分开跑
python -m uvicorn app.api.main:app --host 127.0.0.1 --port 8000
python -m streamlit run app/web/Home.py --server.port 8501
python -m app.workers.continuous --interval 300

# 只手动跑一轮采集 (不发 Telegram)
python scripts/run_live_cycle.py --db data/finance_radar.sqlite3

# 只跑一次本地健康检查, 不联网
python -m app.workers.continuous --once --health-only

# 测试
python -m pytest -q                      # 664 passed, 5 skipped, 20 subtests
python -m compileall -q app scripts tests
```

入口 : Web <http://127.0.0.1:8501> API 文档 <http://127.0.0.1:8000/docs> 健康检查 </api/v1/health>

9. 名词表

名词	大白话解释
P0 / P1 / P2 / P3	来源权威等级。P0 = 官方监管机构（SEC、美联储、CFTC、FDA、FTC、FDIC、ECB...）；P1 = 发行人官方（公司自己发的新闻稿）；P2 = 聚合发现源（线索级）；P3 = 传闻
candidate / verified weak / rejected	事件的四种状态。只有 verified 才有可能进告警队列
EVIDENCE_READY INSUFFICIENT HUMAN_REVIEW	证据 Agent 的三种结论：证据齐了 / 证据不足 / 有冲突需要人来看
exact passage (精确引文)	从原文里抠出来的原话，不是 AI 复述的。这是我们整个产品的立身之本
shadow（影子）	模型在跑、在记录，但结果不影响任何正式判断。我们的风险路由模型一直是这个状态
MISSED_WINDOW	该采行情的那个时间点错过了。如实标记，不拿现在的价格假装是当时的
dry-run	只算不发。Telegram 默认就是这个模式
lease（租约）	数据库里的一把带过期时间的锁，保证同一时刻只有一个采集周期在跑
WAL	SQLite 的一种日志模式，读和写不互相阻塞
outbox（出箱）	先把要发的消息落库，再异步发送。发失败可以重试，而且不会重复发（有唯一键 + 租约）
S / A++ / A / B / C	人工评的事件严重程度。告警只发 S / A++ / A
DEGRADED	部分来源失败但整轮跑完了。不等于 worker 挂了
fail closed	拿不准的时候往“更保守”走。比如没有收据的旧决策一律作废、缺证据一律 INSUFFICIENT
内容寻址	文件按内容的 SHA-256 命名存放。同样内容只存一份，而且事后能验证没被改过

10. 答辩可能被问到的，先自己想清楚

Q：AI 在你们项目里到底干了啥？

两处。一个是 shadow 风险路由分类器（盲测过门了，但因为标签是 AI 打的，仍然不承认它有资格出 shadow）；一个是只写摘要的本地小模型（输出要过 8 道校验，判定全归确定性规则）。证据门、抽取规则、评级、告警资格全是人和规则。AI 参与的是排序和摘要，不是结论。

Q：你们怎么保证不碰交易？

三层。数据库 CHECK 约束（插不进去）、CI 自动扫描交易写路由（提交不进去）、故障注入演练枚举 API 路由断言不存在（跑起来也没有）。

Q：数据是真的吗？

是。生产库 13,357 条事件，来自 13 个官方源。每条都能点回原文，并且有 SHA-256 内容寻址的原始 HTML/PDF 快照存档。

Q：听说你们模型误报 95%？

那是 v1 的数字，已经修好了。当前生产跑的 v4 在 blind-v3 上误报率 6.7%、准确率 96.3%、11 道门全过。完整三代对比见第 3.1 节。

而且我们把 v1 的失败、v2 的失败和修复过程全部留档了：v1 的 40 条逐条错例、v2 的教训一句话（“证据状态是结构化闸门，不是文本类别”）、v4 的架构改动。发现失败 → 公开失败 → 定位根因 → 换架构 → 重新盲测，整条链有据可查。

Q：那 v4 都过门了，为什么还是 shadow？

因为 blind-v2 和 blind-v3 的标签是 AI 按评分规则打的，不是人打的（模型卡里 `human_labels_claimed: false` 写得很清楚）。我们不拿 AI 标签冒充人工标注，所以它只拿到 `QUALIFIED_SHADOW`。要真正晋级，得两个人独立双盲标 80 条、第三人仲裁、重新冻结 blind-v4。

Q：模型对什么都喊风险，把输出反过来不就对了？

实测不行。反过来准确率 50.0% → 47.5%，风险召回 100% → 0%，只是换了一道门不过。因为 v1 是退化（对什么都喊 RISK）不是反相关——它的 ROC-AUC 是 0.857，排序方向本来就是对的，坏的是决策阈值。反转只在 $AUC < 0.5$ 时才有意义。

Q：断网了怎么演示？

4 个冻结回放案例 + 可执行离线终端，完全不依赖网络，也不依赖当天有没有真实重大事件发生。

Q：为什么用 SQLite 不用 PostgreSQL？

单机个人研究场景，WAL 模式下读写不阻塞，够用且零运维。专业性靠持久卷、WAL、租约、幂等出箱、在线备份、隔离恢复验证、健康检查和自动重启来证明，不靠换数据库。PostgreSQL 是留好的扩展门，不是当前重写目标。

Q：那个 `--send` 是干嘛的？

Telegram 真实外发的开关。不加就是 dry-run，一条都不发。演示的时候不要手贱去加。

最后一句，答辩收尾就用这个

“我们没有让模型更自信，而是让系统更会证明、更会拒绝、更能回放。”